StrengthLabs

Documento de Arquitectura del Sistema

MVP — Versión 1.0

Versión	1.0
Estado	Borrador inicial
Fecha	Abril 2026
Plataforma	iOS / Android
Audiencia	Equipo de desarrollo

> Este documento describe la arquitectura técnica del MVP de StrengthLabs, una aplicación móvil para el registro, seguimiento y análisis de entrenamientos físicos. Está orientado a un máximo de 100 usuarios y prioriza la simplicidad, la mantenibilidad y la velocidad de entrega.

Confidencial — Uso interno del equipo

Índice

1. Resumen Ejecutivo **2**

2. Alcance del MVP **2**
- 2.1. Funcionalidades incluidas . . . 2
- 2.2. Funcionalidades excluidas del MVP . . . 2

3. Stack Tecnológico **3**
- 3.1. Resumen general . . . 3

4. Arquitectura del Sistema **3**
- 4.1. Visión general . . . 3
- 4.2. Diagrama de componentes . . . 3
- 4.3. Flujo de petición típico . . . 4

5. Estructura del Backend **4**
- 5.1. Organización de carpetas . . . 4
- 5.2. Módulos principales . . . 5
  - 5.2.1. Motor de Fatiga (`fatigue_engine.py`) . . . 5
  - 5.2.2. Rutinas predeterminadas . . . 5

6. Diseño de Base de Datos **6**
- 6.1. Modelo entidad-relación . . . 6

7. API REST — Endpoints **7**

8. Estructura del Frontend (Flutter) **8**
- 8.1. Organización de carpetas . . . 8
- 8.2. Gestión de estado . . . 8
- 8.3. Navegación . . . 9

9. Seguridad **9**

10. Infraestructura y Despliegue **9**
- 10.1. Configuración del VPS . . . 9
- 10.2. Variables de entorno requeridas . . . 10
- 10.3. Estrategia de despliegue . . . 10

11. Consideraciones de Escalabilidad **10**

12. Decisiones Técnicas Clave **11**

13. Hoja de Ruta Post-MVP **11**

14. Glosario **11**

1. Resumen Ejecutivo

StrengthLabs es una aplicación móvil multiplataforma (iOS y Android) orientada a usuarios de gimnasio de todos los niveles. Su propósito principal es permitir el registro detallado de entrenamientos, calcular indicadores de fatiga y sobreentrenamiento mediante fórmulas basadas en RPE y volumen semanal, y ofrecer rutinas predeterminadas adaptadas al perfil del usuario.

El presente MVP está delimitado intencionalmente para validar el núcleo del producto con hasta **100 usuarios**. La arquitectura elegida favorece la sencillez operativa y el costo reducido de infraestructura, sin sacrificar la capacidad de evolucionar hacia versiones más grandes en el futuro.

2. Alcance del MVP

2.1. Funcionalidades incluidas

- **Autenticación:** registro e inicio de sesión con email y contraseña.
- **Registro de entrenamientos:** sesiones con ejercicios, series, repeticiones, peso y RPE.
- **Historial:** visualización de sesiones pasadas con filtros por fecha y grupo muscular.
- **Cálculo de fatiga:** motor de lógica propia basado en RPE y volumen semanal acumulado.
- **Indicador de sobreentrenamiento:** alerta cuando el volumen/RPE supera umbrales configurados.
- **Rutinas predeterminadas:** catálogo de rutinas base que el sistema sugiere según nivel y objetivo del usuario.
- **Exportación de datos:** descarga del historial en formato Excel (.xlsx) y CSV.

2.2. Funcionalidades excluidas del MVP

- Generación dinámica de rutinas con IA / LLM.
- Integración con dispositivos wearables.
- Social features (seguir usuarios, comparar estadísticas).
- Notificaciones push.
- Panel de administración web.

3. Stack Tecnológico

3.1. Resumen general

Capa	Tecnología	Justificación
Frontend móvil	Flutter 3.x	Cross-platform iOS/Android, un solo codebase, excelente DX
Backend API	Python + FastAPI	Async nativo, alto rendimiento, tipado con Pydantic
Base de datos	PostgreSQL 16	Relacional, robusto, excelente soporte analítico
ORM	SQLAlchemy 2.x	Integración nativa con FastAPI; migraciones con Alembic
Autenticación	JWT (python-jose)	Stateless, simple, seguro para el volumen del MVP
Servidor	VPS Ubuntu 22.04	Costo fijo, control total, suficiente para 100 usuarios
Proxy inverso	Nginx	Manejo de HTTPS y load balancing básico
Exportación	openpyxl / csv	Generación nativa de .xlsx y .csv sin dependencias extra

4. Arquitectura del Sistema

4.1. Visión general

El sistema sigue una arquitectura **cliente-servidor de dos capas** desacopladas. El cliente móvil Flutter se comunica exclusivamente a través de una API REST con el backend FastAPI.

> **Principio de diseño:** Todo el estado de negocio vive en el backend. La app Flutter es únicamente responsable de la presentación y la interacción del usuario.

4.2. Diagrama de componentes

Diagrama de alto nivel

```
+-----------------------------------------------+
```

```
|              VPS (Ubuntu 22.04)                    |
|                                                    |
|  +----------+    +------------------+  +------+   |
|  |  Nginx   |--->|   FastAPI App    |->| PG16 |   |
|  |  (443)   |    |   (Gunicorn)     |  |(5432)|   |
|  +----------+    |                  |  +------+   |
|       ^          |  [Auth JWT    ]  |             |
|       |          |  [Workouts    ]  |             |
|       |          |  [Fatigue     ]  |             |
|       |          |  [Routines    ]  |             |
|       |          |  [Export      ]  |             |
|       |          +------------------+             |
+-------|-------------------------------------------+
|  HTTPS / REST
|
+-------+-----------+
|   Flutter App     |
|   iOS / Android   |
+-------------------+
```

4.3. Flujo de petición típico

1. El usuario interactúa con la app Flutter.
2. Flutter envía una petición HTTPS a Nginx con el JWT en el header `Authorization: Bearer <token>`.
3. Nginx hace proxy al proceso Gunicorn (FastAPI).
4. FastAPI valida el JWT, ejecuta la lógica de negocio y consulta PostgreSQL.
5. La respuesta JSON retorna por el mismo camino a Flutter.
6. Flutter actualiza la UI con los datos recibidos.

5. Estructura del Backend

5.1. Organización de carpetas

Estructura del proyecto FastAPI

```
StrengthLabs-backend/
app/
main.py              # Punto de entrada, registro de routers
config.py            # Variables de entorno (pydantic-settings)
database.py          # Sesion SQLAlchemy
models/
user.py
workout.py
```

```
            exercise.py
            set_log.py
            routine.py
            schemas/
            user.py
            workout.py
            exercise.py
            routine.py
            routers/
            auth.py            # POST /auth/register  /auth/login
            workouts.py        # CRUD de sesiones
            exercises.py       # Catalogo de ejercicios
            routines.py        # Rutinas predeterminadas
            fatigue.py         # GET /fatigue/summary
            export.py          # GET /export/xlsx  /export/csv
            services/
            auth_service.py
            fatigue_engine.py  # Motor de calculo de fatiga
            export_service.py
            core/
            security.py        # Hash bcrypt, JWT utils
            dependencies.py    # get_current_user, get_db
            alembic/               # Migraciones de BD
            tests/
            requirements.txt
            .env
            Dockerfile
```

5.2. Módulos principales

5.2.1. Motor de Fatiga (`fatigue_engine.py`)

El cálculo de fatiga y sobreentrenamiento se basa en las siguientes métricas:

- **Volumen semanal acumulado:** suma total de series por grupo muscular en los últimos 7 días.
- **RPE promedio ponderado:** media ponderada del RPE declarado en cada serie.
- **Índice de fatiga (IF):** `IF = Volumen_semanal * RPE_promedio / 10`
- **Umbral de sobreentrenamiento:** configurable por perfil (principiante: IF > 30; intermedio: IF > 50; avanzado: IF > 70).

5.2.2. Rutinas predeterminadas

Las rutinas se almacenan en la BD como plantillas. El sistema sugiere una rutina basándose en el nivel y el objetivo del usuario:

Nivel	Objetivo	Rutina sugerida
Principiante	Fuerza general	Full Body 3x/semana
Principiante	Hipertrofia	Full Body 3x/semana (volumen)
Intermedio	Fuerza	Push/Pull/Legs 6x/semana
Intermedio	Hipertrofia	Upper/Lower 4x/semana
Avanzado	Fuerza máxima	5/3/1 Wendler
Avanzado	Hipertrofia	PPL 6x/semana (alto volumen)

6. Diseño de Base de Datos

6.1. Modelo entidad-relación

Esquema principal de la BD

```
users
id            UUID         PK
email         VARCHAR      UNIQUE NOT NULL
password_hash VARCHAR      NOT NULL
name          VARCHAR
level         ENUM         (beginner, intermediate, advanced)
goal          ENUM         (strength, hypertrophy, endurance)
created_at    TIMESTAMP

exercises
id            UUID         PK
name          VARCHAR      NOT NULL
muscle_group  VARCHAR      NOT NULL
description   TEXT
is_custom     BOOLEAN      DEFAULT FALSE
user_id       UUID         FK -> users  (NULL si es global)

workout_sessions
id            UUID         PK
user_id       UUID         FK -> users
name          VARCHAR
notes         TEXT
started_at    TIMESTAMP
ended_at      TIMESTAMP

set_logs
id            UUID         PK
session_id    UUID         FK -> workout_sessions
exercise_id   UUID         FK -> exercises
set_number    INTEGER
reps          INTEGER
weight_kg     DECIMAL(6,2)
rpe           DECIMAL(3,1) -- escala 1-10
created_at    TIMESTAMP
```

```
routines
id            UUID        PK
name          VARCHAR     NOT NULL
level         ENUM
goal          ENUM
description   TEXT
days_per_week INTEGER

routine_exercises
id            UUID        PK
routine_id    UUID        FK -> routines
exercise_id   UUID        FK -> exercises
day_of_week   INTEGER     (1-7)
sets          INTEGER
reps_target   VARCHAR     (e.g. "8-12")
order_index   INTEGER
```

7. API REST — Endpoints

Endpoint	Método	Descripción
`/auth/register`	POST	Crea un nuevo usuario.
`/auth/login`	POST	Autentica y devuelve JWT de acceso y refresco.
`/auth/me`	GET	Devuelve el perfil del usuario autenticado.
`/workouts`	GET	Lista sesiones del usuario (paginado, filtros por fecha y músculo).
`/workouts`	POST	Crea una nueva sesión de entrenamiento.
`/workouts/{id}`	GET	Detalle de una sesión con todos sus sets.
`/workouts/{id}`	PUT	Actualiza nombre y notas de la sesión.
`/workouts/{id}`	DELETE	Elimina una sesión.
`/exercises`	GET	Catálogo global más ejercicios del usuario.
`/exercises`	POST	Crea un ejercicio personalizado.
`/fatigue/summary`	GET	Devuelve IF, RPE promedio y alerta de sobreentrenamiento.
`/fatigue/weekly`	GET	Volumen semanal por grupo muscular.
`/routines`	GET	Rutinas sugeridas según nivel y objetivo.
`/routines/{id}`	GET	Detalle de una rutina con sus ejercicios.
`/export/xlsx`	GET	Descarga historial completo en .xlsx.
`/export/csv`	GET	Descarga historial completo en .csv.

8. Estructura del Frontend (Flutter)

8.1. Organización de carpetas

Estructura del proyecto Flutter

```
StrengthLabs-flutter/
lib/
main.dart
app.dart                # MaterialApp, router, theme
core/
constants/            # URLs base, colores, strings
network/              # Dio client, interceptors JWT
storage/              # SecureStorage (token)
router/               # GoRouter - rutas nombradas
features/
auth/
data/               # AuthRepository, RemoteDataSource
domain/             # Entidades, UseCases
presentation/       # LoginPage, RegisterPage, Cubit
workouts/
data/
domain/
presentation/       # WorkoutListPage, WorkoutDetailPage
# ActiveWorkoutPage (registro en vivo)
fatigue/
presentation/       # FatigueDashboardPage
routines/
presentation/       # RoutinesPage, RoutineDetailPage
export/
presentation/       # ExportPage (botones XLSX / CSV)
shared/
widgets/              # Botones, cards, inputs comunes
utils/                # Formatters, validators
assets/
fonts/
icons/
pubspec.yaml
analysis_options.yaml
```

8.2. Gestión de estado

Se utiliza **Flutter Bloc / Cubit** como solución de gestión de estado, por su predictibilidad, testabilidad y separación clara entre UI y lógica.

8.3. Navegación

Se utiliza **GoRouter** para navegación declarativa con soporte de deep links y guardas de autenticación.

9. Seguridad

- **Contraseñas:** hash con `bcrypt` (factor de coste 12) antes de almacenar en BD.
- **Tokens:** JWT firmados con HS256. Access token de 60 minutos; refresh token de 7 días.
- **Transporte:** HTTPS obligatorio. Nginx con certificado TLS vía Let's Encrypt.
- **Autorización:** cada endpoint protegido verifica que el recurso pertenece al usuario autenticado.
- **Variables de entorno:** secretos en `.env`, nunca en el repositorio.
- **CORS:** configurado explícitamente en FastAPI para aceptar solo orígenes de la app móvil.

10. Infraestructura y Despliegue

10.1. Configuración del VPS

Stack de produccion en el VPS

```
VPS (Ubuntu 22.04 LTS)

Nginx  -- proxy inverso, SSL termination
/etc/nginx/sites-enabled/StrengthLabs

StrengthLabs-api  -- proceso Gunicorn (FastAPI)
Workers : 2-4 (segun RAM disponible)
Puerto  : 8000 (interno, solo localhost)
Manager : systemd

PostgreSQL 16
Puerto  : 5432 (solo localhost)
Base    : strengthlabs_prod
```

10.2. Variables de entorno requeridas

Archivo .env (backend)

```
DATABASE_URL=postgresql://user:password@localhost:5432/strengthlabs_prod
SECRET_KEY=<clave_secreta_larga_y_aleatoria>
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
REFRESH_TOKEN_EXPIRE_DAYS=7
```

10.3. Estrategia de despliegue

Para el MVP se utiliza un flujo de despliegue manual simple:

1. `git pull` en el VPS desde la rama `main`.
2. Ejecutar migraciones: `alembic upgrade head`.
3. Reiniciar el servicio: `systemctl restart strengthlabs-api`.
4. Verificar logs: `journalctl -u strengthlabs-api -f`.

En versiones futuras se puede adoptar CI/CD con GitHub Actions.

11. Consideraciones de Escalabilidad

Aunque el MVP está dimensionado para 100 usuarios, la arquitectura está diseñada para escalar sin cambios estructurales mayores:

- **BD:** PostgreSQL soporta cómodamente decenas de miles de usuarios. En caso de crecimiento se puede migrar a RDS o Supabase.
- **API:** FastAPI con Gunicorn permite incrementar workers o migrar a Docker + Compose sin reescribir el código.
- **Exportación:** la generación de archivos pesados puede moverse a tareas asíncronas con Celery + Redis.
- **Caché:** se puede agregar Redis para cachear resultados de fatiga sin modificar la lógica existente.

12. Decisiones Técnicas Clave

Decisión	Alternativa descartada	Razón
Flutter vs React Native	React Native Expo	Performance nativo superior; un solo lenguaje (Dart)
FastAPI vs Django	Django REST Framework	Más ligero, async nativo, ideal para una API enfocada
PostgreSQL vs MongoDB	MongoDB	Datos relacionales; SQL es más adecuado
JWT vs sesiones	Sesiones en Redis	Reduce dependencias; suficiente para 100 usuarios
VPS vs serverless	Vercel + PlanetScale	Costo fijo predecible, sin límites de ejecución

13. Hoja de Ruta Post-MVP

1. **v1.1** — Notificaciones push (Firebase Cloud Messaging).
2. **v1.2** — Generación dinámica de rutinas con IA (LLM vía API).
3. **v1.3** — Integración con wearables (Apple Health, Google Fit).
4. **v2.0** — Panel de administración web para usuarios y rutinas.

14. Glosario

RPE *Rate of Perceived Exertion* — Escala subjetiva del esfuerzo percibido (1–10).

IF Índice de Fatiga — Métrica que combina volumen semanal y RPE promedio para estimar la fatiga acumulada.

Set log
Registro de una serie individual dentro de una sesión de entrenamiento.

JWT
JSON Web Token — Estándar para tokens de autenticación stateless.

ORM
Object-Relational Mapper — Capa de abstracción entre el código Python y la base de datos SQL.

VPS *Virtual Private Server* — Servidor virtual con recursos dedicados alojado en la nube.

CORS
Cross-Origin Resource Sharing — Política de seguridad del cliente para peticiones entre dominios.